

TANDEM HEALTH · PLATFORM TAKE-HOME

Medical Notes API

Design, load testing, and one improvement

Node.js, TypeScript, Fastify, and PostgreSQL

What I built

API

Node.js 24, TypeScript, and Fastify. Three endpoints to create, read, and update medical notes.

Database

PostgreSQL 17 through Postgres.js. Note history is stored as immutable versions.

Local setup

Docker Compose, persistent database volume, request schemas, health, and readiness endpoints.

For the load tests the API ran directly on the host for easier resource sampling. PostgreSQL remained in Docker, so this was still a mixed host/container setup.

Note versions and concurrent updates

PRIMARY KEY (id, version)

Every edit is a new row. PUT reads the current version and inserts the next one in a single statement.

Writer A

next = 5

Writer B

next = 5



one 200
one 409

Current behavior

- No duplicate persisted versions
- No silent overwrite
- Complete revision history
- Works across Node processes

This keeps the stored data consistent. A real product still has to decide whether a stale edit should be rejected, retried, or serialized.

What I tested before running load

Acceptance tests

- POST → GET → PUT → GET
- Response fields and version increments
- Invalid UUIDs and request bodies
- Missing notes

Concurrent updates

- Five updates sent to the same note
- Successful writes have unique versions
- Conflicting writes return 409
- No unexpected 500 responses

I also checked manually that a note survives a service restart.

How the test works

Workload

70%

GET

- 50 seeded notes
- 15 s warm-up + 60 s measured
- 10, 25, 50, 100, 200 VUs
- No think time

20%

POST

10%

PUT

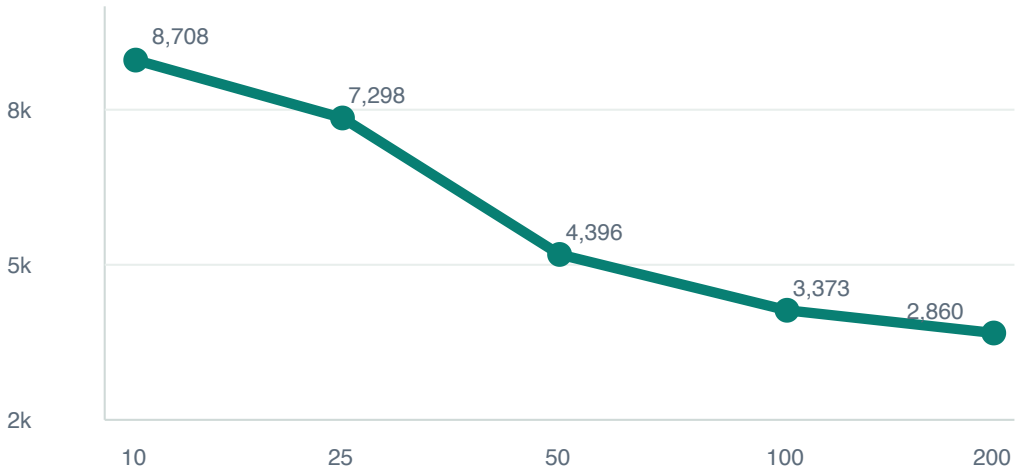
What I measured

- Completed workload requests/s
- p50 / p95 / p99 by operation
- Unexpected failures and expected conflicts
- API CPU and resident memory

This is a local saturation test, not a user simulation. Each VU sends another request as soon as the previous one finishes.

Baseline results

Throughput (requests/s)



25

first observed degradation

0%

unexpected failures

90 ms

GET p95 at 200 VUs

Throughput peaked at the lowest tested level while latency rose from **1.33 ms** to **90.36 ms**.

This showed that the service was saturated. It did not tell me whether the cause was CPU, database connections, or another shared resource.

Testing the pool-size hypothesis

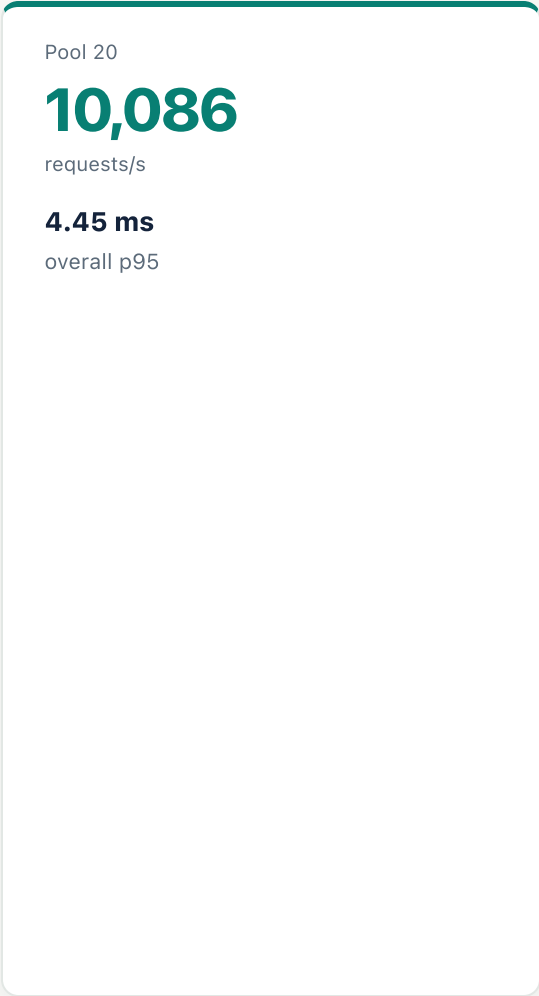
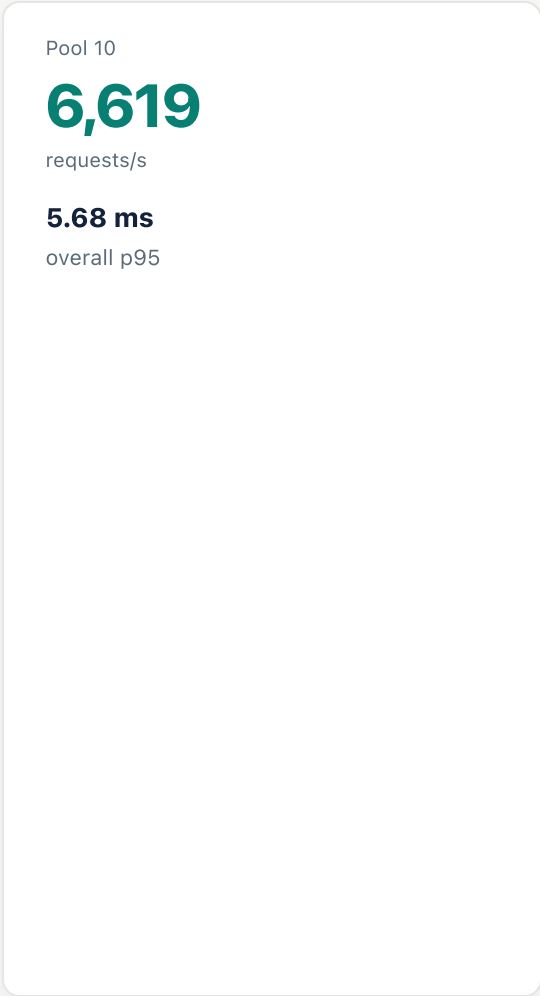
PostgreSQL sampling

One-second pg_stat_activity snapshots often showed idle connections.

The queries take milliseconds, so the sampler could easily miss short periods where every pooled connection was busy. I treated this result as inconclusive.

Pool comparison

I kept the workload at 25 VUs and changed the pool limit from 10 to 20.



Throughput increased by **52.4%** and p95 fell by **21.7%**. That was enough evidence to continue testing pool sizes.

Choosing a pool size

Pool	req/s	overall p95	PUT p95	conflicts/s
20	8,509	9.20 ms	10.65 ms	24
30	8,588	8.65 ms	10.11 ms	25
40	10,028	10.72 ms	14.44 ms	77
50	10,131	11.85 ms	14.61 ms	108

50 VUs, one Node process. Expected 409s are included in request throughput and also tracked separately.

1%
more throughput with pool 50

10
additional possible database connections

Pool 50 also had higher overall latency and more conflicts. I chose pool 40 as the smallest setting close to the measured ceiling.

Pool 40 results

VUs	req/s	GET p50	GET p95	GET p99	Infra errors	Peak CPU	Peak RSS
10	7,393	0.88 ms	2.15 ms	6.49 ms	0%	93%	211 MiB
25	10,305	1.67 ms	3.02 ms	4.04 ms	0%	97%	215 MiB
50	10,159	3.74 ms	6.07 ms	8.48 ms	0%	105%	207 MiB
100	6,342	15.01 ms	20.92 ms	23.00 ms	0%	109%	287 MiB
200	4,199	46.49 ms	63.59 ms	79.29 ms	0%	105%	331 MiB

25 VUs was the highest tested point before degradation. At 50 VUs, throughput was almost unchanged while GET p95 roughly doubled.

CPU and RSS came from two-second ps samples of the API process during the measured 60 seconds.

What changed

VUs	Pool 10 req/s	Pool 40 req/s	Change	Pool 10 GET p95	Pool 40 GET p95
10	8,708	7,393	-15.1%	1.33 ms	2.15 ms
25	7,298	10,305	+41.2%	4.82 ms	3.02 ms
50	4,396	10,159	+131.1%	15.31 ms	6.07 ms
100	3,373	6,342	+88.1%	40.20 ms	20.92 ms
200	2,860	4,199	+46.8%	90.36 ms	63.59 ms

+18%

higher peak measured throughput

25 → 50

first observed degradation

0%

unexpected infrastructure errors

The 10-VU regression did not reproduce in a later repeat, so I treat that low-load difference as run-to-run variation.

Trade-offs, assumptions, and limits

Trade-offs I made

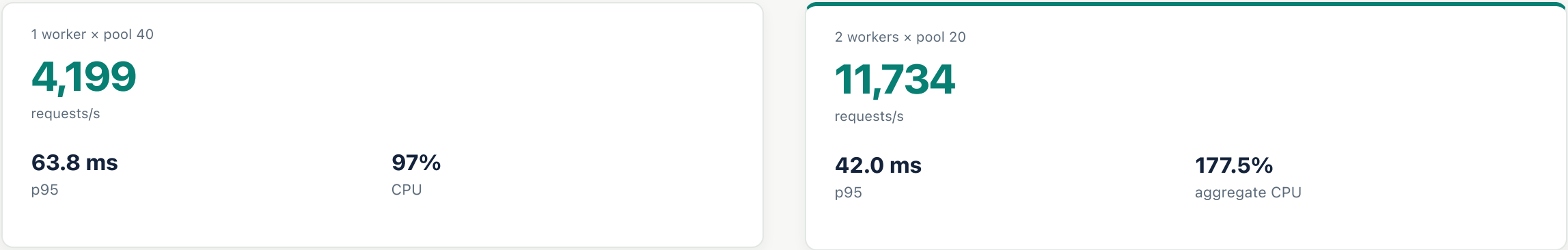
- Append-only versions preserve history, but use more storage and leave stale-edit handling unresolved.
- 409 prevents a silent overwrite, but still represents unsuccessful work for the user.
- Pool 40 favored throughput. Pool 30 had lower latency, fewer conflicts, and used fewer connections.
- The closed-loop workload finds saturation quickly, but is not representative user behavior.

What limits the result

- Small payloads, 50 hot notes, fixed 70/20/10 traffic, and no think time.
- API, k6, and Dockerized PostgreSQL ran on one machine; there was no TLS, ingress, or authentication.
- Most configurations have one 60-second measured run, so small differences may be noise.
- One-second PostgreSQL samples can miss millisecond contention; pool wait and event-loop lag were not measured.
- Throughput includes expected 409s, so it is not identical to successful business work.

The tests support a relative claim about this local setup. They do not establish production capacity, an exact saturation boundary, or a number of real users.

Two Node processes



This was one unpaired run against different database state. Memory increased by 31% and conflicts rose to about 188/s. It suggests a useful next test; it does not prove 2.8× scaling.

What the test says

25 VUs at about 10,300 req/s was the highest tested point before degradation with one process and pool 40.

At 50 VUs, throughput was effectively flat while latency doubled. That is the first observed degradation point in the improved series.

At every tested level, unexpected infrastructure failures remained zero.

Test boundaries

- Small, fixed payloads
- 50 cache-friendly notes
- Local network
- No TLS, authentication, or reverse proxy

What I would do next

- 1 Repeat the one-, two-, and four-process tests while keeping the total database connection limit fixed.
- 2 Measure pool wait time, event-loop lag, query duration, PostgreSQL waits, and each process separately.
- 3 Decide what should happen when two users update the same note before adding automatic retries.
- 4 Add request and query timeouts, bound pool waits, drain connections on shutdown, and test database failures.
- 5 Repeat with realistic note sizes and traffic, separate hosts, and an arrival-rate target tied to an actual latency objective.

DISCUSSION

Questions

The detailed results and scope decisions are in the appendix.

Update conflicts

Ten percent of workload requests update one of only 50 seeded notes. Larger pools and more workers allow more of those updates to reach PostgreSQL concurrently.

A 409 is an expected, correctness-preserving response in this design. It is still unsuccessful business work and must not disappear inside aggregate throughput.

~16%

estimated PUT conflict share in the two-worker 200-VU experiment

This is why the next step is to define edit semantics before adding retries or serialization.

Decisions made for this exercise

Authentication

authorId is attribution supplied by the client, not verified identity.

Production topology

No ingress, TLS, separate hosts, autoscaling, or managed database.

Product semantics

No decision yet on stale-edit reconciliation, idempotency, retention, or deletion.

These are scope decisions for the take-home, not suggested production defaults.